Farmer RAG Advisory System

AI-Powered Agricultural Guidance for Smallholder Farmers

CMU 04-801-W3 · Agentic AI: Fundamentals and Applications

Glorry Sibomana & Ethel Kwagalakwe

The Problem

Smallholder farmers in East Africa need personalized, actionable advice — when to plant, how to control pests, when to harvest — but lack access to reliable guidance.

A single LLM call is not enough because:

- **Multi-source synthesis** : "Should I plant maize now?" requires weather + market prices + agronomic knowledge *combined*
- **Adaptive behavior** : if retrieval is weak, re-retrieve; if a tool fails, retry
- **Verification** : hallucinated dosages or planting schedules can cause real harm
- **Memory** : farmers ask follow-ups across sessions ("Remember the fertilizer advice?")

> An **agent** is the right abstraction: it reasons, calls tools, critiques itself, and remembers.

Agent Architecture

```
Farmer Query
    │
    ▼
┌─────────────┐     ┌──────────────────────────────────────┐
│  PLANNING   │ ──▶ │  EXECUTION (Tools)                   │
│  (Reasoning)│     │ Weather · Market · Knowledge Base    │
└─────────────┘     └──────────────────────────────────────┘
    ▲                                │
    │                                ▼
    │               ┌───────────────────────────────┐
    └────────────── │ CRITIQUE (Verification)       │
  re-retrieve if    │ groundedness < 0.80 → retry   │
  score too low     └───────────────────────────────┘
                                     │
                                     ▼
                      Response + Source Citations
                      Stored in PostgreSQL (memory)
```

LangGraph graph: `START → reasoning → [execute_tool | verify] → [reasoning | END]`

Tools & Memory

Tools

Tool	Backend	Purpose
`get_weather_forecast`	Open-Meteo API	Real-time weather for planting decisions
`query_agricultural_knowledge`	pgvector + ChromaDB	Retrieval from ingested agronomic docs
`get_market_prices`	Mock (extensible)	Price guidance for market timing

Two-Tier Memory

Layer	Storage	What's kept
Short-term	`AgentState` (in-memory)	Current run: tool calls, sources, verifications
Persistent	PostgreSQL	Conversations, farms/crops, advisories, tool call logs

Pruning: archive after 90 days · summarize conversations > 50 messages

True Agentic Behavior

The agent exhibits all four pillars:

- **Autonomy** — chooses which tools to call and when to stop (no fixed pipeline)
- **Multi-step reasoning** — ReAct loop: *Observe* → *Reason* → *Decide* → *Act* → *Evaluate* → *Update*
- **Adaptive control** — re-retrieves on low groundedness · retries on tool failure · detects infinite loops
- **Persistent memory** — loads conversation history and farmer profile (farms, crops, growth stage) at each session start

```
[REASONING] User asks about fertilizer for maize
[TOOL_CALL] query_agricultural_knowledge(query="fertilizer maize")
[TOOL_RESULT] Found 3 relevant documents
[REASONING] Need weather info for timing
[TOOL_CALL] get_weather_forecast(lat=-1.94, lon=29.87)
[TOOL_RESULT] 20% rain probability next 3 days
[VERIFICATION] 4 claims extracted, 4 supported → groundedness: 1.00
```

Demo

Scenario: "My maize is at flowering stage. What should I do about pests and when should I harvest?"

What to watch for:

1. Agent calls `query_agricultural_knowledge` (multi-part query → multiple sources)
2. Streaming response with **source citations**
3. Groundedness score shown in logs (`>= 0.80` = passes)
4. Follow-up: *"Tell me more about drying"* → agent uses **conversation history**

Evaluation Results

6 test cases · run 2026-02-19

Test	Query	Groundedness	Tools	Passed
kb_001	Recommended spacing for maize?	1.00	1.00	✅
kb_002	Recommended spacing for beans?	0.50	1.00	❌
kb_003	Recommended spacing for tomatoes?	0.50	1.00	✅
kb_004	Weather in Kigali today?	1.00	1.00	✅
kb_005	When to harvest maize + drying?	1.00	1.00	✅
complex_001	Maize planting area requirements	1.00	1.00	✅

5/6 passed · Avg groundedness 0.83 · Avg tool accuracy 1.00

Failure (kb_002): beans spacing not sufficiently covered in ingested docs → verifier strict

Design Trade-offs & Limitations

Design choices

- Role-based nodes (Planning / Execution / Critique) → modularity and debuggability over a single monolithic prompt
- Two-tier memory → cross-session continuity without storing every intermediate state
- Verification step → reduces hallucinations at the cost of latency

Known limitations

- Chunks lack crop/topic metadata → filtered retrieval falls back to unfiltered (lower confidence)
- Verifier can be strict → correct answers occasionally scored below threshold
- KB quality is bounded by ingested documents

Scalability

- At 1,000 users: DB connection pool and sync tool execution fail first → fix with async I/O + read replicas
- High-stakes domain: need human-in-the-loop for critical recommendations (pesticide dosage)

Thank You

Questions?

Farmer RAG · CMU 04-801-W3 · 2026